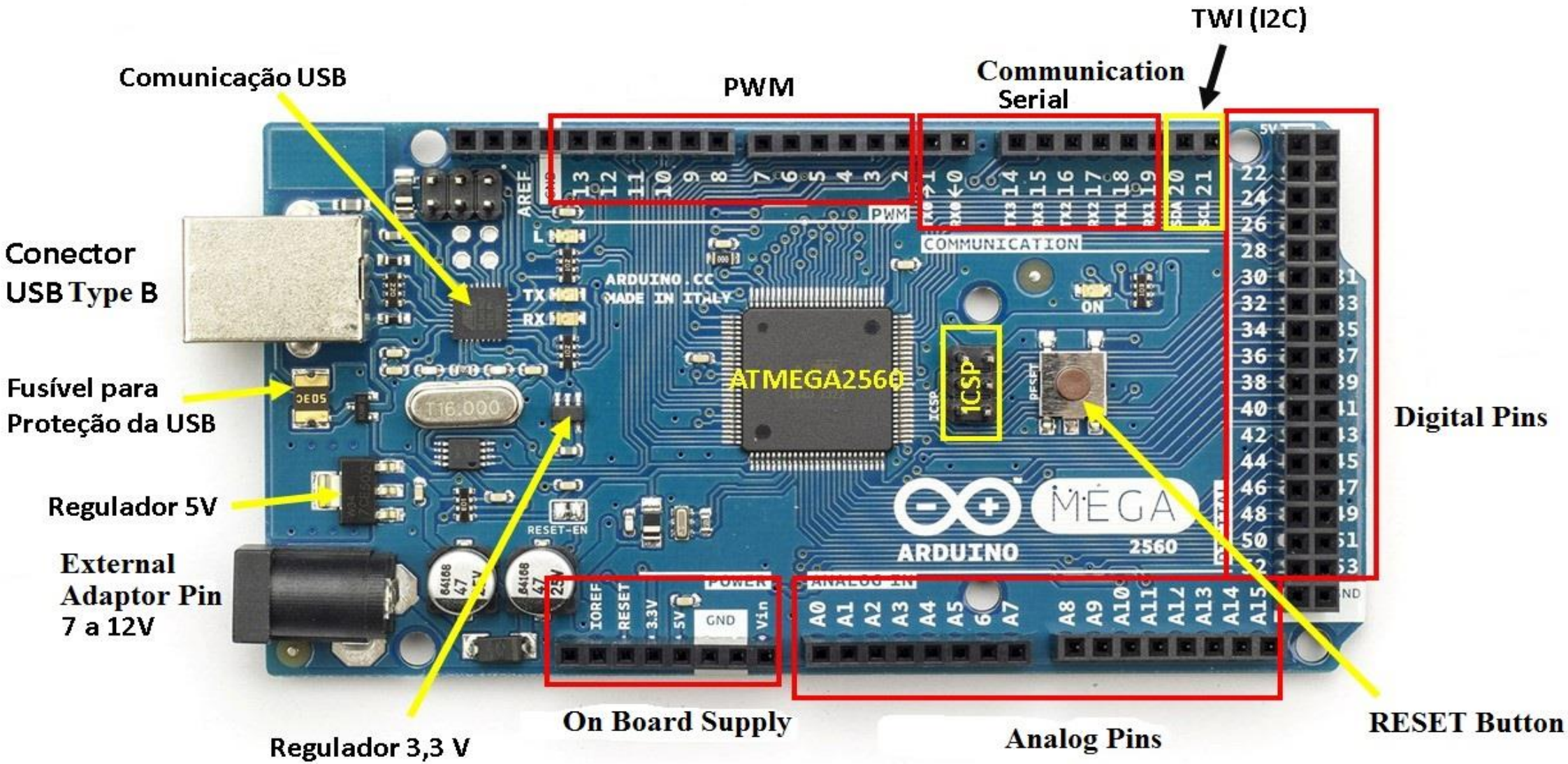
the avr microcontroller and embedded system

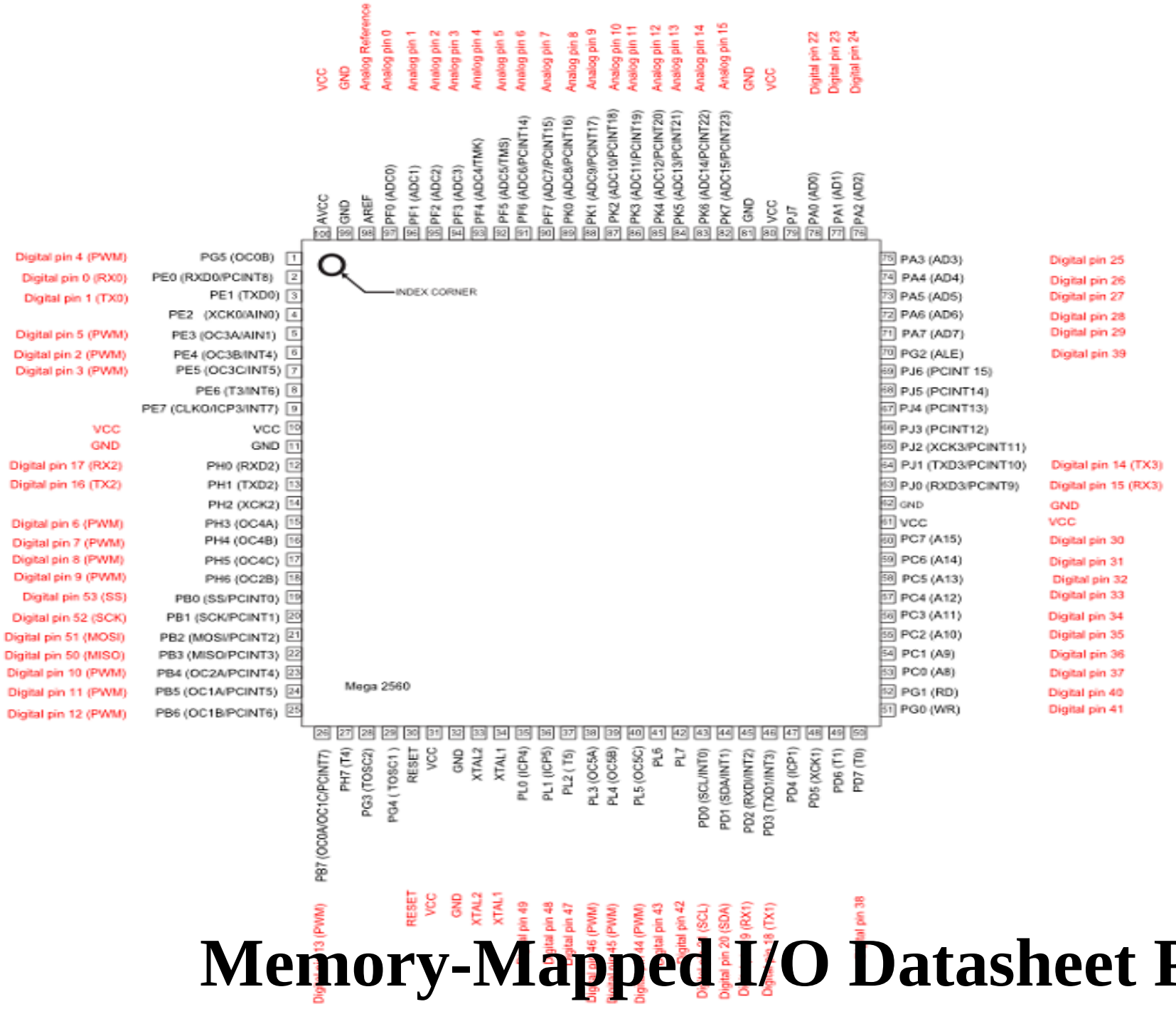
using assembly and c

MUHAMMAD ALI MAZIDI
SARMAD NAIMI
SEPEHR NAIMI

Arduino Mega 2560



Pin Mapping Atmega2560



Memory-Mapped I/O Datasheet Page: 399

Arduino Mega 2560

Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	54 (of which 15 provide PWM output)
Analog Input Pins	16
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz
Length	101.52 mm
Width	53.3 mm
Weight	37 g

AVR Microcontroller I/O PORT Programming:

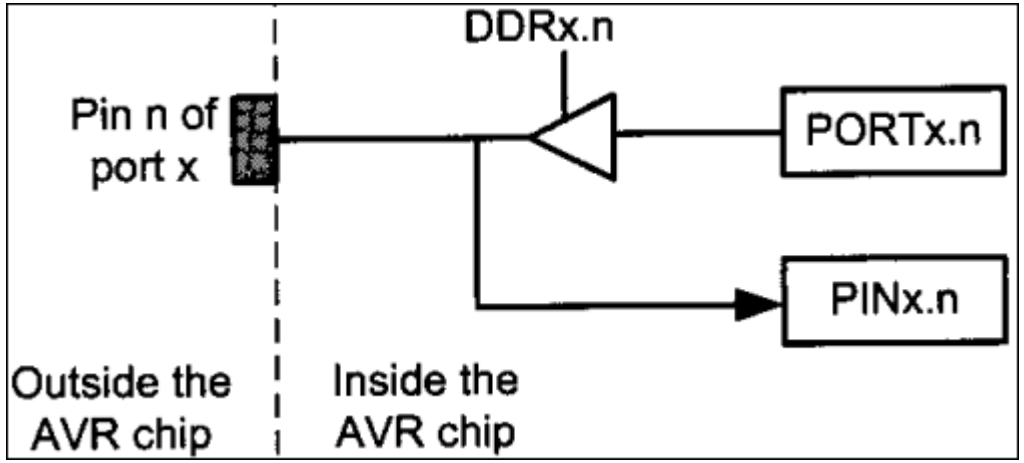
PORTX	Number of Pins
PORTA	8-bit bi-directional I/O port (PA7..PA0)
PORTB	8-bit bi-directional I/O port (PB7..PB0)
PORTC	8-bit bi-directional I/O port (PC7..PC0)
PORTD	8-bit bi-directional I/O port (PD7..PD0)
PORTE	8-bit bi-directional I/O port (PE7..PE0)
PORTF	8-bit bi-directional I/O port (PF7..PF0)
	Analog inputs to the A/D Converter
PORTG	6-bit bi-directional I/O port (PG5..PG0)
PORTH	8-bit bi-directional I/O port (PH7..PH0)
PORTJ	8-bit bi-directional I/O port (PJ7..PJ0)
PORTK	8-bit bi-directional I/O port (PK7..PK0)
	Analog inputs to the A/D Converter
PORTL	8-bit bi-directional I/O port (PL7..PL0)

- **PORTs** must be programmed before used as an **INPUT(s)** or an **OUTPUT(s)**
- **PORTs** have some other functions as ADC, Timers, Counters, Interrupts, Serial Communications, etc.

- Each **PORT** has three **I/O registers**
  - ✓ **DDRx (D**ata **D**irection **R**egister, **DDRB)**
  - ✓ **PORTx (PORTB)**
  - ✓ **PINx (P**ort **IN**put pins**, PINB)**

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0x05 (0x25)	PORTB	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
0x04 (0x24)	DDRB	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
0x03 (0x23)	PINB	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0

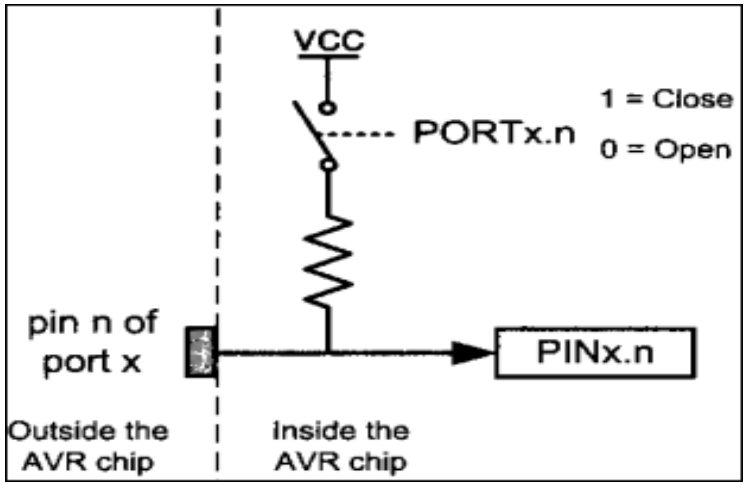
- **DDRx** is used to assign (set) the port as an **INPUT** or an **OUTPUT**
  - ➢ For INPUT DDRBx pin = 0 (0b0000_0000)
  - ➢ For OUTPUT DDRBx pin = 1 (0b1111_1111)



- ❑ Port(s) must be initialized as an input or an output in start of the code.
- ❑ To set port B as an output, DDRBx port bits must be all ONEs.
- ❑ Otherwise **data will not go** from **port register** to the **pins of AVR**.
- ❑ **Upon reset**, DDRX register become zero (**Reset makes port as an INPUT, 0x00**)

Role of PINx and PORTx Registers:

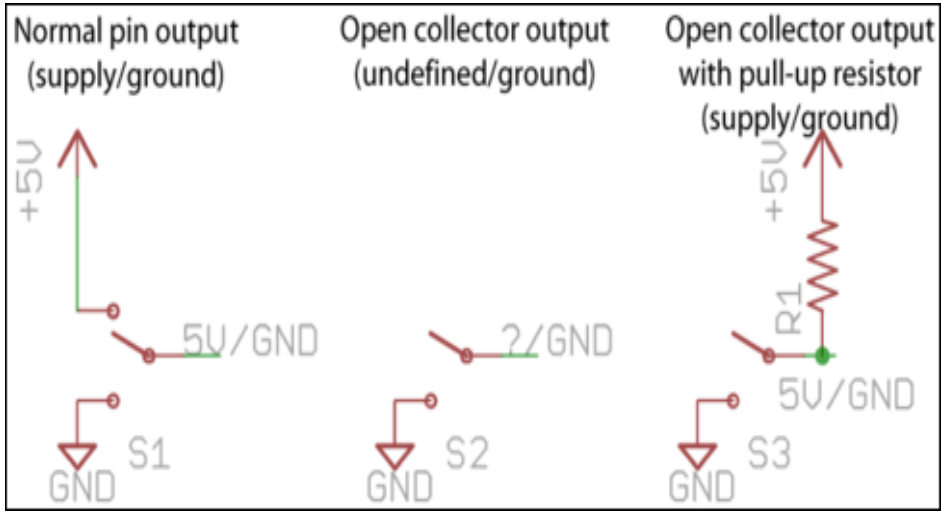
- ❑ To **read data present at the pins**, we should read the **PINx** register
  (To bring data into CPU from pins we read contents of the **PINx** register)
- ❑ To **send data out to pins** we use the **PORTx** register



Internal PULL-UPs of AVR:

To use internal pull-ups for INPUT.

- ✓ DDRx register must be zero
- ✓ PORTx register set to 0xFF (to use internal pull-ups)



- ❑ AVR Microcontroller has Four valued Logic

✓ 0
✓ 1
✓ X (Unknown, Don't Care)
✓ Z (High Impedance or Open Circuit, Floating state, uncertain state)

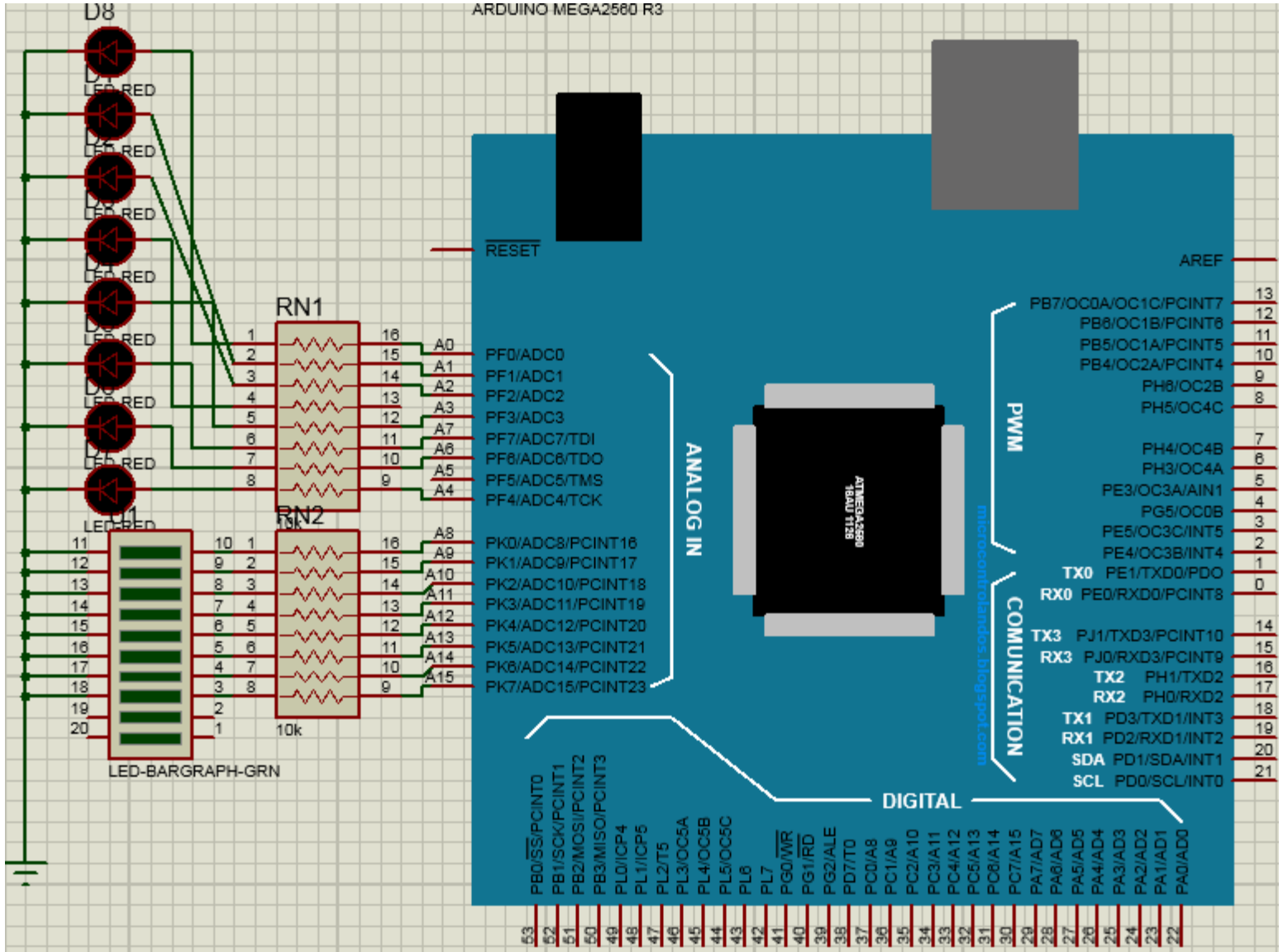
C Data Types for the AVR:

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
char	8-bit	–128 to +127
unsigned int	16-bit	0 to 65,535
int	16-bit	–32,768 to +32,767
unsigned long	32-bit	0 to 4,294,967,295
long	32-bit	–2,147,483,648 to +2,147,483,648
float	32-bit	±1.175e-38 to ±3.402e38
double	32-bit	±1.175e-38 to ±3.402e38

I/O PORT Programming:

* Write an AVR program to **send** values **0x00** to **PORTF** and **0xFF** to **PORTK**. **OR**
* Suppose LEDs are connected on PORTF and PORTK of µc, OFF all LEDs of PORTF and ON all LEDs of PORTK

```
#include <avr/io.h>
int main (void)
{
        DDRF = 0xFF;
        DDRK = 0xFF;
        while(1)
        {
                PORTF  = 0x00;
                PORTK  = 0xFF;
        }
        return  0;
}
```



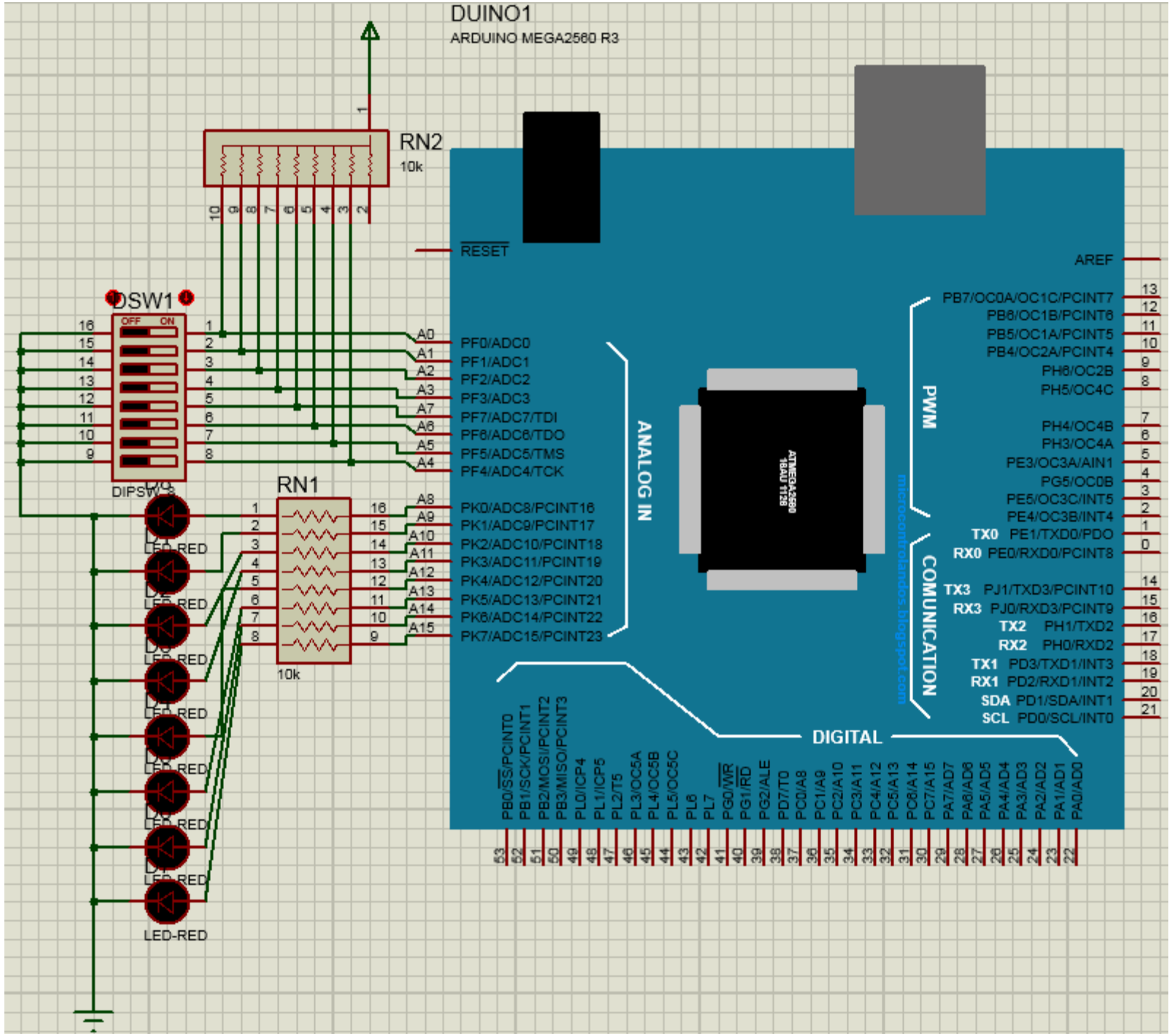
* Write a program to get a byte of data from PORTF, and then send it to PORTK.
* Suppose SWITCHs are connected on PORTF and LEDS are connected on PORTK of µc, send SWITCHs (PORTF) condition/status/data to LEDs (PORTK).

```
#include <avr/io.h>

int main (void)
{
        DDRK = 0xFF;          // OUTPUT
        DDRF = 0x00;          // INPUT
        while(1)
        {
                PORTK  = PINF;
        }
        return  0;
}
```

```
#include <avr/io.h>

int main (void)
{
        unsigned char temp;
        DDRK = 0xFF;          // OUTPUT
        DDRF = 0x00;          // INPUT
        while(1)
        {
                temp   = PINF;
                PORTK  = temp;
        }
        return  0;
}
```


DUINO1
ARDUINO MEGA2560 R3
RN2
10k
DSW1
DIPSW8
RN1
10k
LED-RED
RESET
AREF
ANALOG IN
PWM
COMUNICATION
DIGITAL
ATMEGA2560
16AU 1128
microcontrolandos.blogspot.com
A0 PF0/ADC0
A1 PF1/ADC1
A2 PF2/ADC2
A3 PF3/ADC3
A7 PF7/ADC7/TDI
A6 PF6/ADC6/TDO
A5 PF5/ADC5/TMS
A4 PF4/ADC4/TCK
A8 PK0/ADC8/PCINT16
A9 PK1/ADC9/PCINT17
A10 PK2/ADC10/PCINT18
A11 PK3/ADC11/PCINT19
A12 PK4/ADC12/PCINT20
A13 PK5/ADC13/PCINT21
A14 PK6/ADC14/PCINT22
A15 PK7/ADC15/PCINT23
PB7/OC0A/OC1C/PCINT7
PB6/OC1B/PCINT6
PB5/OC1A/PCINT5
PB4/OC2A/PCINT4
PH6/OC2B
PH5/OC4C
PH4/OC4B
PH3/OC4A
PE3/OC3A/AIN1
PG5/OC0B
PE5/OC3C/INT5
PE4/OC3B/INT4
TX0 PE1/TXD0/PDO
RX0 PE0/RXD0/PCINT8
TX3 PJ1/TXD3/PCINT10
RX3 PJ0/RXD3/PCINT9
TX2 PH1/TXD2
RX2 PH0/RXD2
TX1 PD3/TXD1/INT3
RX1 PD2/RXD1/INT2
SDA PD1/SDA/INT1
SCL PD0/SCL/INT0
PB0/SS/PCINT0
PB1/SCK/PCINT1
PB2/MOSI/PCINT2
PB3/MISO/PCINT3
PL0/ICP4
PL1/ICP5
PL2/T5
PL3/OC5A
PL4/OC5B
PL5/OC5C
PL6
PL7
PG0/WR
PG1/RD
PG2/ALE
PD7/T0
PC0/A8
PC1/A9
PC2/A10
PC3/A11
PC4/A12
PC5/A13
PC6/A14
PC7/A15
PA7/AD7
PA6/AD6
PA5/AD5
PA4/AD4
PA3/AD3
PA2/AD2
PA1/AD1
PA0/AD0

Time Delay

- ✓ By using a for loop
- ✓ By using predefined C functions (library)
- ✓ By using AVR timer registers

- ❑ Crystal frequency is the important factor in calculating time delays
- ❑ Complier used to compile the C program (code)

Write a program to toggle PORTK continuously with a 250ms delay. Crystal frequency is 16MHz.

```
#include <avr/io.h>
void delay500ms(void)
{
        unsigned int i;
        for(i=0;i<7936;i++);
}
int main (void){
        DDRK = 0xFF;
        while(1)
        {
                PORTK  = 0xFF;
                delay500ms();
                 PORTK  = 0x00;
                delay500ms();
        }
        return  0;
}
```

```
#include <avr/io.h>
#include<util/delay.h>
int main (void)
{
        DDRK = 0xFF;
        while(1)
        {
                PORTK  = 0xFF;
                _delay_ms(250);
                PORTK  = 0x00;
                _delay_ms(250);
        }
        return  0;
}
```

LEDs are connected to pins of PORTK. Write a program that shows the count from 00H to FFH (0000 0000 to 1111 1111) on the LEDs.

```
#include <avr/io.h>
#include<util/delay.h>

int main (void)
{
	DDRK = 0xFF;
	while(1)
	{
		PORTK++;
		_delay_ms(200);
	}
	return  0;
}
```

I/O PORT Bit-wise Programming:

Bit-wise Operators in C:

AND (&), OR (|), EX-OR (^), INVERTER (~), SHIFT RIGHT (>>), SHIFT LEFT (<<)

Bit-wise Logic Operators for C

		AND	OR	EX-OR	Inverter
A	B	A&B	A|B	A^B	Y=~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	

```
0x35 & 0x0F = 0x05        /* ANDing */
0x04 | 0x68 = 0x6C        /* ORing  */
0x54 ^ 0x78 = 0x2C        /* XORing */
~0x55 = 0xAA              /* Inverting 55H */
```

Bit-wise shift operation in C:

There are two bit-wise shift operators.

Table 7-4: Bit-wise Shift Operators for C

Operation	Symbol	Format of Shift Operation
Shift right	>>	data >> number of bits to be shifted right
Shift left	<<	data << number of bits to be shifted left

Examples of shift operators in C:

1. 0b00010000 >> 3 = 0b00000010 //shifting right 3 times
2. 0b00010000 << 3 = 0b10000000 //shifting left 3 times
3. 1 << 3 = 0b00001000 //shifting left 3 times

Bit-wise shift operation and bit manipulation:

For improving code clarity;
Instead of writing “0b00100000”, we can write “0b00000001 << 5” or simply “1 << 5”
OR
Sometimes we need numbers like 0b11101111. To generate such a number, first shift then invert it.
“0b11101111” can be written as “~ (1<<4)”

Write code to generate the following numbers:

(a) A number that has only a one in position D7
(b) A number that has only a one in position D2
(c) A number that has only a one in position D4
(d) A number that has only a zero in position D5
(e) A number that has only a zero in position D3
(f) A number that has only a zero in position D1

Solution:

(a) `(1<<7)`
(b) `(1<<2)`
(c) `(1<<4)`
(d) `~(1<<5)`
(e) `~(1<<3)`
(f) `~(1<<1)`

* Write a program to toggle only bit 4 of PORTK continuously without disturbing the rest of the pins of PORTK. (PORTK bit 4 is connected to LED)
* Write a program to toggle only bit 4 of PORTK continuously without disturbing the rest of the pins of PORTK. (PORTK bit 4 is connected to LED)

```
#include <avr/io.h>
#include<util/delay.h>
int main (void)
{
        DDRK &= ~(1<<4);   // OUTPUT
        while(1)
        {
                PORTK  |= (1<<4);     //set bit 4 of PORTK
                _delay_ms(100);
                 PORTK  &= ~(1<<4);            //set clear 4 of PORTK
                _delay_ms(100);
        }
        return  0;
}
```

Write a program to get the status of bit 5 of PORTF and send it to bit 7 of PORTK continuously.

```
#include <avr/io.h>

int main (void)
{
        DDRF &= ~(1<<5) ;   // pin 5 of PORTF is INPUT
        DDRK |= (1<<7);       // pin 7 of PORTK is OUTPUT
        while(1)
        {
                if (PINF & (1<<5))                //check pin 5 of PINF
                PORTK |= (1<<7);      //set pin 7 of PORTK
                else
                PORTK &= ~(1<<7); //clear pin 7 of PORTK
        }
        return  0;
}
```

A door sensor is connected to bit 1 of PORTF, and LED (buzzer) is connected to bit 7 of PORTK. Write a program to monitor door sensor and, when it opens, turn on the LED (buzzer).

```
#include <avr/io.h>

int main (void)
{
	DDRF &= ~(1<<1);    // pin 1 of PORTF is INPUT
	DDRK |= (1<<7);       // pin 7 of PORTK is OUTPUT
	while(1)
	{
		if (PINF & (1<<1))                //check pin 5 of PINF
		PORTK |= (1<<7);      //set pin 7 of PORTK
		else
		PORTK &= ~(1<<7); //clear pin 7 of PORTK
	}
	return  0;
}
```

A door sensor is connected to bit 1 of PORTF, and LED (buzzer) is connected to bit 7 of PORTK. Write a program to monitor door sensor and, when it opens, turn on the LED (buzzer).

```
#include <avr/io.h>
#define SENSOR 1
#define LED 7
int main (void)
{
        DDRF &= ~(1<<SENSOR);  // pin 1 of PORTF is INPUT
        DDRK |= (1<<LED);              // pin 7 of PORTK is OUTPUT
        while(1)
        {
                if (PINF & (1<<SENSOR))              //check pin 1 of PINF
                        PORTK |= (1<<LED);           //set pin 7 of PORTK
                else
                        PORTK &= ~(1<<LED);        //clear pin 7 of PORTK
        }
        return  0;
}
```

To generate more complicated numbers, simply OR the numbers.

Example: one in position B7 and another in B4.

(1<<7)|(1<<4)

Write a program to read pins 0 and 1of PORTF, and issues a value (data or number) to PORTK according to the following table:

pin 1	Pin 0	Value
0	0	0x0F
0	1	0x55
1	0	0xAA
1	1	0xF0

```
#include <avr/io.h>
int main (void)
{
        unsigned char z;
        DDRK = 0xFF;
        DDRF = 0x00;
        while(1)
        {
                z = PINF;
                z = z & 0b00000011;
                switch (z)
                {
                        case (0):
                        {
                                PORTK = 0x0F;
                                break;
                        }
                        case (1):
                        {
                                PORTK = 0x55;
                                break;
                        }
                        case (2):
                        {
                                PORTK = 0xAA;
                                break;
                        }
                        case (3):
                        {
                                PORTK = 0xF0;
                                break;
                        }
                }
        }
        return  0;
}
```

Write a program to read pins 0 and 1of PORTF, and issues a value (data or number) to PORTK
according to the following table:

pin 1	Pin 0	Value
0	0	0x0F
0	1	0x55
1	0	0xAA
1	1	0xF0

```
#include<avr/io.h>
int main()
{
  DDRK = 0xFF;
  DDRF &= ~(1<<0);
  DDRF &= ~(1<<1);
  while(1)
  {
    if(!(PINF & (1<<0)) && !(PINF & (1<<1)))
      PORTK = 0x0F;
    if((PINF & (1<<0)) && !(PINF & (1<<1)))
      PORTK = 0x55;
    if(!(PINF & (1<<0)) && (PINF & (1<<1)))
      PORTK = 0xAA;
    if((PINF & (1<<0)) && (PINF & (1<<1)))
      PORTK = 0xF0;
  }
  return 0;
}
```

IF PIN is not high

```
#include<avr/io.h>
int main()
{
  DDRK = 0xFF;
  DDRF &= (1<<0);
  while(1)
  {
    if( !(PINF & (1<<0)))
      PORTK = 0xff;
     else
      PORTK = 0x55;
  }
  return 0;
}
```

IF both PINs are high

```
#include<avr/io.h>
int main()
{
  DDRK = 0xFF;
  DDRF &= (1<<0);
  DDRF &= (1<<1);
  while(1)
  {
    if((PINF & (1<<0)) && (PINF & (1<<1)))
      PORTK = 0xff;
     else
      PORTK = 0x55;
  }
  return 0;
}
```

IF both PINs are Low

```
#include<avr/io.h>
int main()
{
  DDRK = 0xFF;
  DDRF &= (1<<0);
  DDRF &= (1<<1);
  while(1)
  {
    if(!(PINF & (1<<0)) && !(PINF & (1<<1)))
      PORTK = 0xff;
     else
      PORTK = 0x55;
  }
  return 0;
}
```

IF PIN0 is Low and other is high

```
#include<avr/io.h>
int main()
{
  DDRK = 0xFF;
  DDRF &= (1<<0);
  DDRF &= (1<<1);
  while(1)
  {
    if(!(PINF & (1<<0)) && (PINF & (1<<1)))
      PORTK = 0xff;
     else
      PORTK = 0x55;
  }
  return 0;
}
```